

Attention Is All You Need

(注意力机制是你所需要的一切)

Ashish Vaswani [*]	Noam Shazeer [*]	Niki Parmar [*]	Jakob Uszkoreit [*]
Google Brain	Google Brain	Google Research	Google Research
avaswani@google.com	noam@google.com	nikip@google.com	usz@google.com
Llion Jones [*]	Aidan N. Gomez ^{*†}		
Google Research	University of Toronto (多伦多大学)		
llion@google.com	aidan@cs.toronto.edu		
Lukasz Kaiser [*]	Illia Polosukhin ^{*‡}		
Google Brain	illia.polosukhin@gmail.com		
lukaszkaiser@google.com			

在提供恰当署名的前提下，Google 特此授权为新闻或学术作品之用途复制本文中的表格与图。

摘要

主流的序列转导 (sequence transduction) 模型基于包含编码器 (encoder) 和解码器 (decoder) 的复杂循环 (recurrent) 或卷积 (convolutional) 神经网络。表现最好的模型还通过一种注意力机制 (attention mechanism) 将编码器和解码器连接起来。我们提出一种新的、简单的网络架构——Transformer，它完全基于注意力机制，彻底摒弃了循环和卷积。在两项机器翻译任务上的实验表明，这些模型在质量上更优，同时具有更好的可并行性，并且训练所需时间显著减少。我们的模型在 WMT 2014 英语到德语翻译任务上取得了 28.4 BLEU 的成绩，比包括集成模型在内的现有最佳结果提高了 2 BLEU 以上。在 WMT 2014 英语到法语翻译任务上，我们的模型在 8 块 GPU 上训练 3.5 天后，取得了 41.8 的单模型最新最优 (state-of-the-art) BLEU 分数，而其训练成本仅为文献中最佳模型的一小部分。我们还表明，通过将 Transformer 成功应用于英语成分句法分析 (constituency parsing) ——无论训练数据规模大或小——它能够很好地泛化到其他任务上。

^{*} 同等贡献。署名顺序为随机。Jakob 提出用自注意力 (self-attention) 替换 RNN，并发起了对这一想法的评估工作。Ashish 与 Illia 一起设计并实现了第一批 Transformer 模型，并在本工作的方方面面都发挥了关键作用。Noam 提出了缩放点积注意力 (scaled dot-product attention)、多头注意力 (multi-head attention) 以及无参数的位置表示，并成为几乎参与了每一处细节的另一位核心人员。Niki 在我们最初的代码库以及 tensor2tensor 中设计、实现、调优并评估了数不清的模型变体。Llion 也对新的模型变体进行了实验，负责了我们最初的代码库，以及高效推理与可视化。Lukasz 和 Aidan 花费了无数个漫长的日子，设计并实现了 tensor2tensor 的各个部分，替换了我们早期的代码库，极大地改善了结果并大幅加速了我们的研究。

[†] 此工作在 Google Brain 任职期间完成。 [‡] 此工作在 Google Research 任职期间完成。

第 31 届神经信息处理系统大会 (NIPS 2017), 美国加利福尼亚州长滩。

1 引言

循环神经网络 (recurrent neural networks), 尤其是长短期记忆网络 (long short-term memory) [13] 和门控循环 (gated recurrent) [7] 神经网络, 已被牢固确立为序列建模和转导问题 (如语言建模和机器翻译) 中的最新最优方法 [35, 2, 5]。此后, 大量工作持续推动着循环语言模型和编码器-解码器架构的边界 [38, 24, 15]。

循环模型通常沿输入和输出序列的符号位置来分解计算。它们将位置与计算时间中的步骤对齐, 生成一个隐藏状态序列 h_t , 其中 h_t 是前一隐藏状态 h_{t-1} 和位置 t 的输入的函数。这种固有的顺序特性使得训练样本内部无法并行化, 而在序列长度较长时这一点变得至关重要, 因为内存约束限制了跨样本的批处理。近期工作通过因子分解技巧 [21] 和条件计算 [32] 在计算效率上取得了显著提升, 后者还同时提升了模型性能。然而, 顺序计算这一根本约束依然存在。

注意力机制已成为各种任务中引人注目的序列建模和转导模型不可或缺的组成部分, 它允许对依赖关系进行建模而无需考虑它们在输入或输出序列中的距离 [2, 19]。然而, 除少数情况 [27] 外, 这类注意力机制都是与循环网络结合使用的。

在本工作中, 我们提出 Transformer, 这是一种摒弃了循环、转而完全依赖注意力机制来刻画输入与输出之间全局依赖关系的模型架构。Transformer 允许显著更高层次的并行化, 并且在 8 块 P100 GPU 上仅训练十二小时后即可达到翻译质量上的最新最优水平。

2 背景

减少顺序计算这一目标也构成了 Extended Neural GPU [16]、ByteNet [18] 和 ConvS2S [9] 的基础, 它们都使用卷积神经网络作为基本构建模块, 对所有输入和输出位置并行地计算隐藏表示。在这些模型中, 将来自任意两个输入或输出位置的信号关联起来所需的操作数随着位置间的距离而增长, 对于 ConvS2S 是线性增长, 对于 ByteNet 是对数增长。这使得学习远距离位置之间的依赖关系更加困难 [12]。在 Transformer 中, 这一数量被减少为一个常数级别的操作数, 尽管代价是由于对注意力加权后的位置进行平均而导致有效分辨率下降——我们用第 3.2 节中描述的多头注意力来抵消这一影响。

自注意力 (self-attention), 有时称为内部注意力 (intra-attention), 是一种关联单个序列中不同位置以计算该序列表示的注意力机制。自注意力已成功应用于多种任务, 包括阅读理解、抽象式摘要、文本蕴含以及学习与任务无关的句子表示 [4, 27, 28, 22]。

端到端记忆网络基于一种循环注意力机制而非序列对齐的循环, 并已被证明在简单语言问答和语言建模任务上表现良好 [34]。

然而, 据我们所知, Transformer 是第一个完全依赖自注意力来计算其输入和输出表示, 而不使用

序列对齐的 RNN 或卷积的转导模型。在接下来的章节中，我们将描述 Transformer，阐明使用自注意力的动机，并讨论它相对于 [17, 18] 和 [9] 等模型的优势。

3 模型架构

大多数具有竞争力的神经序列转导模型都具有编码器-解码器结构 [5, 2, 35]。其中，编码器将符号表示的输入序列 $(x_1, \dots, x_n)$ 映射到连续表示序列 $z = (z_1, \dots, z_n)$ 。给定 z ，解码器随后一次生成一个元素，逐步生成符号的输出序列 $(y_1, \dots, y_m)$ 。在每一步，模型都是自回归的 (auto-regressive) [10]，在生成下一个符号时将先前生成的符号作为额外输入。

Transformer 遵循这一整体架构，在编码器和解码器中都使用堆叠的自注意力以及逐点 (point-wise) 的全连接层，分别如图 1 的左半部分和右半部分所示。

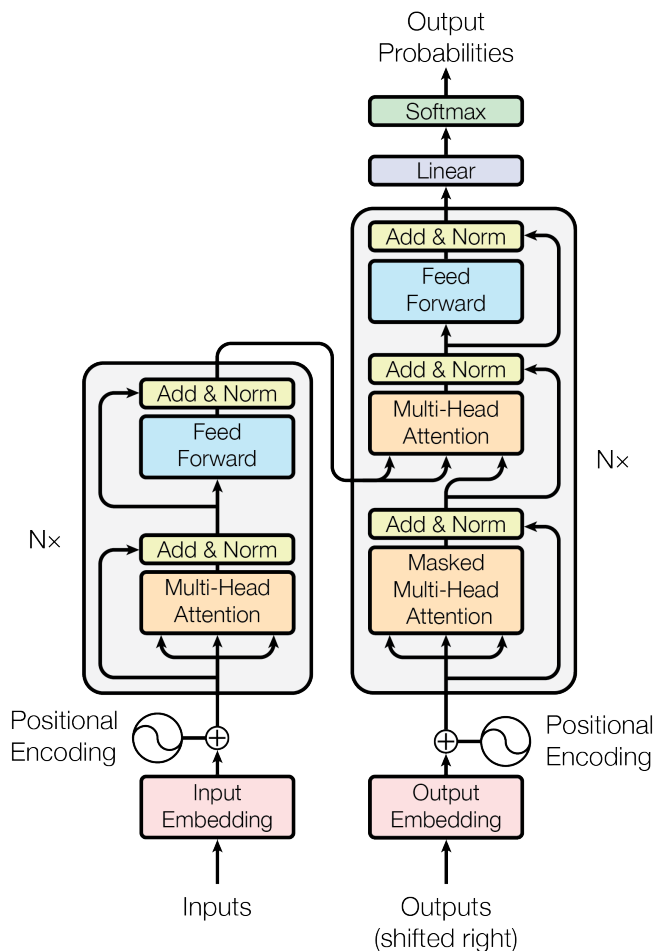


图 1: Transformer——模型架构。

3.1 编码器与解码器堆栈

编码器：编码器由 $N = 6$ 个相同的层堆叠而成。每一层有两个子层。第一个是多头自注意力机制，第二个是一个简单的、逐位置（position-wise）的全连接前馈网络。我们在两个子层中的每一个周围都采用残差连接（residual connection）[11]，其后接层归一化（layer normalization）[1]。也就是说，每个子层的输出为 $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中 $\text{Sublayer}(x)$ 是该子层自身实现的函数。为便于这些残差连接，模型中的所有子层以及嵌入层都产生维度为 $d_{\text{model}} = 512$ 的输出。

解码器：解码器同样由 $N = 6$ 个相同的层堆叠而成。除了每个编码器层中的两个子层之外，解码器还插入了第三个子层，该子层对编码器堆栈的输出执行多头注意力。与编码器类似，我们在每个子层周围采用残差连接，其后接层归一化。我们还修改了解码器堆栈中的自注意力子层，以防止某个位置关注到其后续位置。这种掩码（masking），结合输出嵌入偏移一个位置的事实，确保了对位置 i 的预测只能依赖于位置小于 i 处的已知输出。

3.2 注意力

一个注意力函数可以描述为：将一个查询（query）和一组键值对（key-value pairs）映射到一个输出，其中查询、键、值和输出都是向量。输出被计算为值的加权和，其中分配给每个值的权重由查询与对应键的兼容性函数（compatibility function）计算得出。



图 2：（左）缩放点积注意力。（右）多头注意力由多个并行运行的注意力层组成。

3.2.1 缩放点积注意力

我们将我们这种特定的注意力称为“缩放点积注意力”（Scaled Dot-Product Attention，图 2）。输入由维度为 d_k 的查询和键，以及维度为 d_v 的值组成。我们计算查询与所有键的点积，将每个点积除以 $\sqrt{d_k}$ ，并应用一个 softmax 函数以获得值上的权重。

在实践中，我们将一组查询同时打包成一个矩阵 Q ，从而同时计算注意力函数。键和值也分别被打包成矩阵 K 和 V 。我们将输出矩阵计算为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

两种最常用的注意力函数是加性注意力 (additive attention) [2] 和点积 (乘性) 注意力。除了缩放因子 $\frac{1}{\sqrt{d_k}}$ 之外, 点积注意力与我们的算法完全相同。加性注意力使用一个具有单隐藏层的前馈网络来计算兼容性函数。虽然两者在理论复杂度上相似, 但点积注意力在实践中要快得多, 也更节省空间, 因为它可以使用高度优化的矩阵乘法代码来实现。

虽然对于较小的 d_k 值两种机制表现相似, 但对于较大的 d_k 值, 在不进行缩放的情况下加性注意力优于点积注意力 [3]。我们猜测, 对于较大的 d_k 值, 点积在数量级上会变大, 将 softmax 函数推入其梯度极小的区域¹。为了抵消这一影响, 我们将点积乘以 $\frac{1}{\sqrt{d_k}}$ 进行缩放。

3.2.2 多头注意力

我们发现, 与其使用 d_{model} 维的键、值和查询执行单个注意力函数, 不如将查询、键和值分别用不同的、学习得到的线性投影线性地投影 h 次, 投影到 d_k 、 d_k 和 d_v 维更有益。在这些投影后的查询、键和值的每一个版本上, 我们随后并行执行注意力函数, 产生 d_v 维的输出值。这些输出值被拼接 (concatenate) 起来并再次投影, 得到最终的值, 如图 2 所示。

多头注意力允许模型在不同位置共同关注来自不同表示子空间的信息。若使用单个注意力头, 平均操作会抑制这一点。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

其中 $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

其中投影是参数矩阵 $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ 以及 $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ 。

在本工作中, 我们采用 $h = 8$ 个并行的注意力层, 即头 (heads)。对于其中每一个, 我们使用 $d_k = d_v = d_{\text{model}}/h = 64$ 。由于每个头的维度降低, 总计算成本与使用完整维度的单头注意力相近。

3.2.3 注意力在我们模型中的应用

Transformer 以三种不同的方式使用多头注意力:

- 在“编码器-解码器注意力”层中, 查询来自前一个解码器层, 而记忆中的键和值来自编码器的输出。这使得解码器中的每个位置都能关注输入序列中的所有位置。这模仿了序列到序列模型 (如 [38, 2, 9]) 中典型的编码器-解码器注意力机制。
- 编码器包含自注意力层。在一个自注意力层中, 所有的键、值和查询都来自同一处, 在此情形下即编码器中前一层的输出。编码器中的每个位置都能关注编码器前一层中的所有位置。

¹为说明点积为何会变大, 假设 q 和 k 的各分量是均值为 0、方差为 1 的独立随机变量。那么它们的点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的均值为 0、方差为 d_k 。

- 类似地，解码器中的自注意力层允许解码器中的每个位置关注解码器中直到并包括该位置在内的所有位置。我们需要防止解码器中向左的信息流动，以保持自回归特性。我们在缩放点积注意力内部实现这一点，方法是将 softmax 输入中所有对应于非法连接的值掩去（设为 $-\infty$ ）。参见图 2。

3.3 逐位置前馈网络

除注意力子层外，我们编码器和解码器中的每一层还包含一个全连接前馈网络，它被分别且相同地应用于每个位置。该网络由两个线性变换组成，中间有一个 ReLU 激活。

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

虽然线性变换在不同位置上是相同的，但它们在层与层之间使用不同的参数。另一种描述方式是将其视为两个核大小为 1 的卷积。输入和输出的维度为 $d_{\text{model}} = 512$ ，内层的维度为 $d_{\text{ff}} = 2048$ 。

3.4 嵌入与 Softmax

与其他序列转导模型类似，我们使用学习得到的嵌入将输入 token 和输出 token 转换为维度为 d_{model} 的向量。我们还使用常规的、学习得到的线性变换和 softmax 函数将解码器输出转换为预测的下一个 token 的概率。在我们的模型中，我们在两个嵌入层和 softmax 之前的线性变换之间共享同一个权重矩阵，与 [30] 类似。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{\text{model}}}$ 。

表 1: 不同层类型的最大路径长度、每层复杂度以及最少顺序操作数。 n 是序列长度， d 是表示维度， k 是卷积的核大小， r 是受限自注意力中邻域的大小。

层类型	每层复杂度	顺序操作数	最大路径长度
自注意力	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
循环	$O(n \cdot d^2)$	$O(n)$	$O(n)$
卷积	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
自注意力（受限）	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

3.5 位置编码

由于我们的模型不包含循环和卷积，为了使模型能够利用序列的顺序，我们必须注入一些关于 token 在序列中相对或绝对位置的信息。为此，我们在编码器和解码器堆栈底部将“位置编码”（positional encodings）加到输入嵌入上。位置编码具有与嵌入相同的维度 d_{model} ，因此两者可以相加。位置编码有许多种选择，有学习得到的，也有固定的 [9]。

在本工作中，我们使用不同频率的正弦和余弦函数：

$$PE_{(pos, 2i)} = \sin\left(pos/10000^{2i/d_{\text{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(pos/10000^{2i/d_{\text{model}}}\right)$$

其中 pos 是位置， i 是维度。也就是说，位置编码的每一个维度对应一个正弦曲线。波长构成从 2π 到 $10000 \cdot 2\pi$ 的几何级数。我们选择这个函数是因为我们假设它能让模型容易地学会按相对位置进行关注，因为对于任意固定偏移 k ， PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数。

我们还尝试改用学习得到的位置嵌入 [9]，发现这两个版本产生的结果几乎相同（见表 3 的 (E) 行）。我们选择正弦版本，因为它可能允许模型外推到比训练期间遇到的序列长度更长的序列。

4 为何使用自注意力

在本节中，我们将自注意力层的各个方面与常用于将一个可变长度的符号表示序列 $(x_1, \dots, x_n)$ 映射到另一个等长序列 $(z_1, \dots, z_n)$ （其中 $x_i, z_i \in \mathbb{R}^d$ ，例如典型序列转导编码器或解码器中的一个隐藏层）的循环层和卷积层进行比较。为说明我们使用自注意力的动机，我们考虑三个期望特性。

其一是每层的总计算复杂度。其二是可并行化的计算量，以所需的最少顺序操作数来衡量。

其三是网络中长程依赖之间的路径长度。学习长程依赖是许多序列转导任务中的一项关键挑战。影响学习此类依赖能力的一个关键因素是前向和后向信号必须在网络中穿越的路径长度。输入和输出序列中任意位置组合之间的这些路径越短，就越容易学习长程依赖 [12]。因此，我们也比较由不同层类型组成的网络中任意两个输入和输出位置之间的最大路径长度。

如表 1 所示，自注意力层以常数级别的顺序执行操作数连接所有位置，而循环层需要 $O(n)$ 次顺序操作。在计算复杂度方面，当序列长度 n 小于表示维度 d 时，自注意力层比循环层更快，而这在机器翻译中最新最优模型所使用的句子表示（如词片 word-piece [38] 和字节对 byte-pair [31] 表示）中通常都成立。为提升涉及很长序列的任务的计算性能，可以将自注意力限制为只考虑输入序列中以相应输出位置为中心、大小为 r 的邻域。这会将最大路径长度增加到 $O(n/r)$ 。我们计划在未来的工作中进一步研究这一方法。

一个核宽度为 $k < n$ 的单卷积层并不能连接所有的输入和输出位置对。要做到这一点，在连续核的情形下需要堆叠 $O(n/k)$ 个卷积层，在空洞卷积 (dilated convolutions) [18] 的情形下需要 $O(\log_k(n))$ 个，这会增加网络中任意两个位置之间最长路径的长度。卷积层通常比循环层更昂贵，相差一个因子 k 。然而，可分离卷积 (separable convolutions) [6] 大幅降低了复杂度，降到 $O(k \cdot n \cdot d + n \cdot d^2)$ 。但即便在 $k = n$ 的情况下，一个可分离卷积的复杂度也等于一个自注意力层和一个逐点前馈层的组合，而这正是我们在模型中采用的方法。

作为附带的好处，自注意力可能产生更具可解释性的模型。我们检查了来自模型的注意力分布，并在附录中给出和讨论了一些示例。不仅各个注意力头清晰地学会了执行不同的任务，许多头似乎还表现出与句子的句法和语义结构相关的行为。

5 训练

本节描述我们模型的训练机制。

5.1 训练数据与批处理

我们在标准的 WMT 2014 英语-德语数据集上进行训练，该数据集包含约 450 万个句对。句子使用字节对编码 (byte-pair encoding) [3] 进行编码，它具有约 37000 个 token 的共享源-目标词表。对于英语-法语，我们使用了规模显著更大的 WMT 2014 英语-法语数据集，它包含 3600 万个句子，并将 token 拆分为一个 32000 词片的词表 [38]。句对按近似的序列长度被批处理到一起。每个训练批次包含一组句对，约含 25000 个源 token 和 25000 个目标 token。

5.2 硬件与训练计划

我们在一台配有 8 块 NVIDIA P100 GPU 的机器上训练我们的模型。对于使用本文所述超参数的基础 (base) 模型，每个训练步耗时约 0.4 秒。我们将基础模型总共训练了 100,000 步，即 12 小时。对于我们的大 (big) 模型（在表 3 底行描述），每步耗时 1.0 秒。大模型训练了 300,000 步 (3.5 天)。

5.3 优化器

我们使用了 Adam 优化器 [20]，参数为 $\beta_1 = 0.9$ 、 $\beta_2 = 0.98$ 以及 $\epsilon = 10^{-9}$ 。我们在训练过程中按以下公式变化学习率：

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

这相当于在前 $warmup_steps$ 个训练步中线性地增加学习率，此后则按步数的平方根的倒数成比例地减小学习率。我们使用 $warmup_steps = 4000$ 。

5.4 正则化

我们在训练期间采用三种类型的正则化：

残差 Dropout 我们对每个子层的输出应用 dropout [33]，然后再将其加到子层输入上并进行归一化。此外，我们对编码器和解码器堆栈中嵌入与位置编码之和应用 dropout。对于基础模型，我们使用 $P_{drop} = 0.1$ 的比率。

标签平滑 (Label Smoothing) 在训练期间，我们采用了取值 $\epsilon_{ls} = 0.1$ 的标签平滑 [36]。由于模型学会了变得更不确定，这会损害困惑度 (perplexity)，但能提升准确率和 BLEU 分数。

表 2: Transformer 在英语到德语和英语到法语的 newstest2014 测试集上取得了比以往最新最优模型更好的 BLEU 分数，而训练成本仅为其一小部分。

模型	BLEU		训练成本 (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

6 结果

6.1 机器翻译

在 WMT 2014 英语到德语翻译任务上，大型 Transformer 模型（表 2 中的 Transformer (big)）比此前报告的最佳模型（包括集成模型）高出超过 2.0 BLEU，确立了 28.4 的最新最优 BLEU 分数。该模型的配置列于表 3 的底行。训练在 8 块 P100 GPU 上耗时 3.5 天。即便是我们的基础模型，也超越了此前发表的所有模型和集成模型，而训练成本仅为任何一个有竞争力模型的一小部分。

在 WMT 2014 英语到法语翻译任务上，我们的大模型取得了 41.0 的 BLEU 分数，超越了此前发表的所有单模型，而训练成本不到此前最新最优模型的 1/4。用于英语到法语的 Transformer (big) 模型使用了 dropout 比率 $P_{drop} = 0.1$ ，而非 0.3。

对于基础模型，我们使用通过对最后 5 个检查点（每隔 10 分钟写入一次）取平均得到的单个模型。对于大模型，我们对最后 20 个检查点取平均。我们使用了束搜索（beam search），束大小为 4，长度惩罚 $\alpha = 0.6$ [38]。这些超参数是在开发集上实验后选定的。我们将推理期间的最大输出长度设为输入长度 + 50，但在可能时提前终止 [38]。

表 2 总结了我们的结果，并将我们的翻译质量和训练成本与文献中的其他模型架构进行了比较。我们通过将训练时间、所用 GPU 数量以及对每块 GPU 持续单精度浮点运算能力的估计相乘，来估算训练一个模型所用的浮点运算数²。

²我们对 K80、K40、M40 和 P100 分别采用 2.8、3.7、6.0 和 9.5 TFLOPS 的数值。

6.2 模型变体

为评估 Transformer 不同组成部分的重要性，我们以不同方式改变了基础模型，并在开发集 newstest2013 上测量了英语到德语翻译性能的变化。我们使用了前一节描述的束搜索，但未进行检查点平均。我们在表 3 中给出这些结果。

在表 3 的 (A) 行中，我们改变注意力头的数量以及注意力键和值的维度，同时按第 3.2.2 节所述保持计算量不变。虽然单头注意力比最佳设置差 0.9 BLEU，但头过多时质量也会下降。

在表 3 的 (B) 行中，我们观察到减小注意力键的大小 d_k 会损害模型质量。这表明确定兼容性并非易事，而比点积更复杂的兼容性函数可能更有益。我们在 (C) 行和 (D) 行中进一步观察到，正如预期，更大的模型更好，并且 dropout 对避免过拟合 (over-fitting) 非常有帮助。在 (E) 行中，我们用学习得到的位置嵌入 [9] 替换我们的正弦位置编码，观察到与基础模型几乎相同的结果。

表 3: Transformer 架构的各种变体。未列出的值与基础模型相同。所有指标均在英语到德语翻译开发集 newstest2013 上。所列困惑度是按词片计的, 依据我们的字节对编码, 不应与按词计的困惑度相比较。

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
(D)			4096							4.75	26.2	90
							0.0			5.77	24.6	
							0.2			4.95	25.5	
							0.0			4.67	25.3	
(E)							0.2			5.47	25.7	
		以位置嵌入替代正弦曲线								4.92	25.7	
big	6	1024	4096	16			0.3		300K	4.33	26.4	213

6.3 英语成分句法分析

为评估 Transformer 能否泛化到其他任务, 我们在英语成分句法分析上进行了实验。该任务带来了特定的挑战: 输出受到强烈的结构约束, 并且显著长于输入。此外, RNN 序列到序列模型在小数据环境下一直无法达到最新最优结果 [37]。

我们在 Penn Treebank [25] 的华尔街日报 (Wall Street Journal, WSJ) 部分 (约 4 万个训练句子) 上训练了一个 $d_{\text{model}} = 1024$ 的 4 层 Transformer。我们还在半监督设置下训练它, 使用了规模更大、来自约 1700 万个句子的高置信度语料和 BerkeleyParser 语料 [37]。对于仅 WSJ 的设置, 我们使用了 16K token 的词表; 对于半监督设置, 我们使用了 32K token 的词表。

我们仅进行了少量实验，在 Section 22 开发集上选择 dropout（包括注意力和残差两方面，见第 5.4 节）、学习率和束大小，所有其他参数则与英语到德语的基础翻译模型保持不变。在推理期间，我们将最大输出长度增加到输入长度 + 300。对于仅 WSJ 和半监督两种设置，我们都使用了束大小 21 和 $\alpha = 0.3$ 。

表 4 中我们的结果表明，尽管缺乏针对任务的调优，我们的模型表现得出奇地好，取得了比此前报告的所有模型都更好的结果——除了递归神经网络文法（Recurrent Neural Network Grammar）[8]。

与 RNN 序列到序列模型 [37] 相比，即便仅在 4 万句的 WSJ 训练集上训练，Transformer 也优于 BerkeleyParser [29]。

表 4: Transformer 能很好地泛化到英语成分句法分析（结果基于 WSJ 的 Section 23）。

句法分析器	训练	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	仅 WSJ, 判别式	88.3
Petrov et al. (2006) [29]	仅 WSJ, 判别式	90.4
Zhu et al. (2013) [40]	仅 WSJ, 判别式	90.4
Dyer et al. (2016) [8]	仅 WSJ, 判别式	91.7
Transformer (4 layers)	仅 WSJ, 判别式	91.3
Zhu et al. (2013) [40]	半监督	91.3
Huang & Harper (2009) [14]	半监督	91.3
McClosky et al. (2006) [26]	半监督	92.1
Vinyals & Kaiser et al. (2014) [37]	半监督	92.1
Transformer (4 layers)	半监督	92.7
Luong et al. (2015) [23]	多任务	93.0
Dyer et al. (2016) [8]	生成式	93.3

7 结论

在本工作中，我们提出了 Transformer，这是第一个完全基于注意力的序列转导模型，它用多头自注意力替换了编码器-解码器架构中最常用的循环层。

对于翻译任务，Transformer 的训练速度可以显著快于基于循环层或卷积层的架构。在 WMT 2014 英语到德语和 WMT 2014 英语到法语两项翻译任务上，我们都达到了新的最新最优水平。在前一项任务中，我们的最佳模型甚至超越了此前报告的所有集成模型。

我们对基于注意力的模型的未来感到振奋，并计划将它们应用于其他任务。我们计划将 Transformer 扩展到涉及文本之外的输入和输出模态的问题上，并研究局部的、受限的注意力机制，以高效处理图像、音频和视频等大规模输入和输出。让生成过程减少顺序性是我们的另一个研究目标。

我们用于训练和评估模型的代码可在 <https://github.com/tensorflow/tensor2tensor> 获取。

致谢 我们感谢 Nal Kalchbrenner 和 Stephan Gouws 富有成效的评论、修正和启发。

参考文献

1. Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.
2. Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. CoRR, abs/1409.0473, 2014.
3. Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. CoRR, abs/1703.03906, 2017.
4. Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. arXiv preprint arXiv:1601.06733, 2016.
5. Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. CoRR, abs/1406.1078, 2014.
6. Francois Chollet. Xception: Deep learning with depthwise separable convolutions. arXiv preprint arXiv:1610.02357, 2016.
7. Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. CoRR, abs/1412.3555, 2014.
8. Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In Proc. of NAACL, 2016.
9. Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. arXiv preprint arXiv:1705.03122v2, 2017.
10. Alex Graves. Generating sequences with recurrent neural networks. arXiv preprint arXiv:1308.0850, 2013.
11. Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 770–778, 2016.
12. Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
13. Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. Neural computation, 9(8):1735–1780, 1997.

14. Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.
15. Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.
16. Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems*, (NIPS), 2016.
17. Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.
18. Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.
19. Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.
20. Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
21. Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.
22. Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.
23. Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Łukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.
24. Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.
25. Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.
26. David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.
27. Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.
28. Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.

29. Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.
30. Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.
31. Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
32. Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
33. Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
34. Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.
35. Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.
36. Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.
37. Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.
38. Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
39. Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.
40. Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

附录：注意力可视化

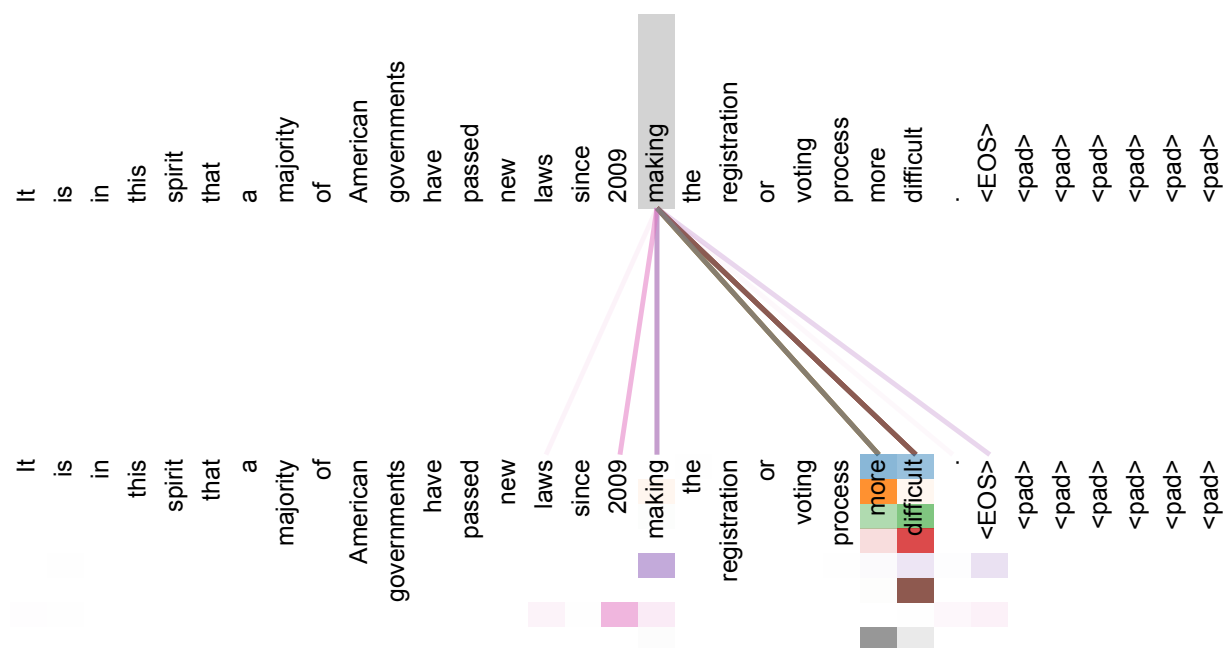


图 3: 在第 5 层（共 6 层）的编码器自注意力中，注意力机制跟踪长距离依赖关系的一个示例。许多注意力头都关注到动词‘making’的一个远距离依赖项，从而补全了短语‘making...more difficult’。此处仅展示词‘making’的注意力。不同颜色代表不同的头。建议以彩色查看。

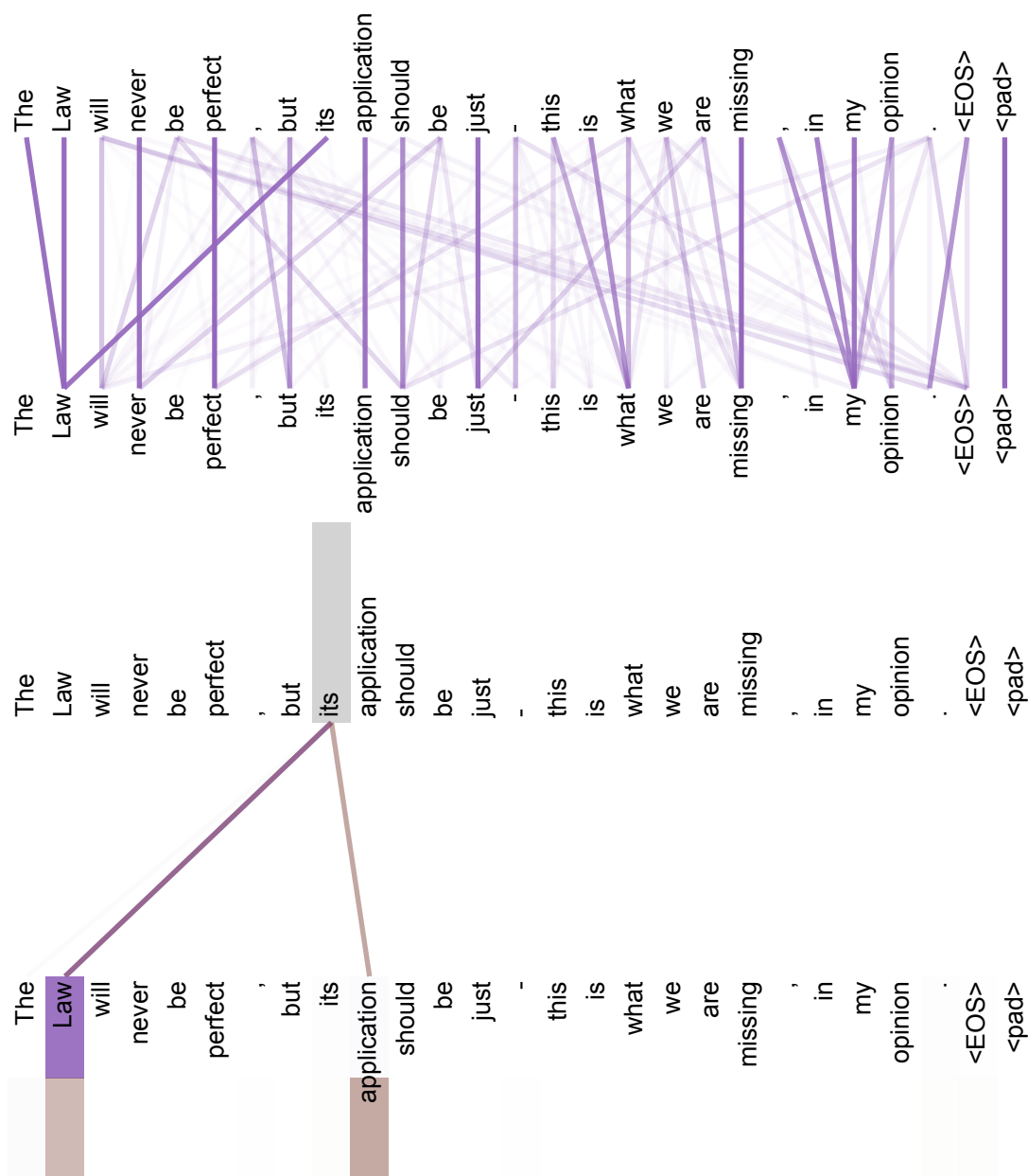


图 4: 同样位于第 5 层 (共 6 层) 的两个注意力头, 显然参与了指代消解 (anaphora resolution)。上图: 第 5 个头的完整注意力。下图: 仅针对词 'its' 的、来自第 5 和第 6 个注意力头的孤立注意力。注意对于这个词, 注意力非常尖锐。

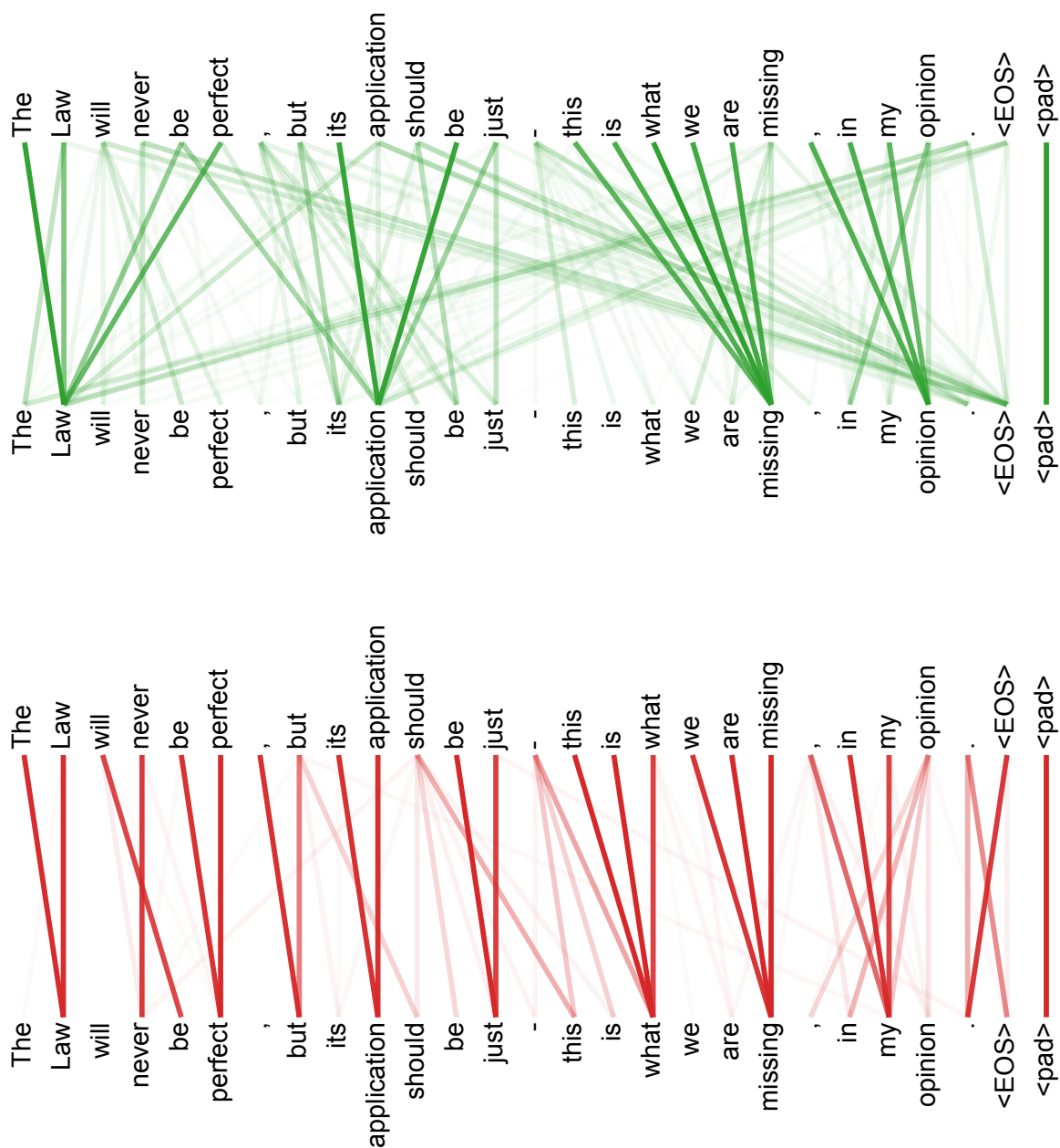


图 5: 许多注意力头表现出似乎与句子结构相关的行为。我们在上方给出两个这样的示例, 来自第 5 层 (共 6 层) 编码器自注意力中两个不同的头。这些头清晰地学会了执行不同的任务。